DAR ES SALAAM INSTITUTE OF TECHNOLOGY (D.I.T)

DEPARTMENT OF ELECTRICAL ENGINEERING

PROGRAM: ELECTRICAL AND RENEWABLE ENERGY TECHNOLOGY

NAME: NOELA LUPEJA MASOLWA -24032322624

CLASS: 0D24 ERE

ACADEMIC YEAR: 2025/2026

MODULE: FUNDAMENTAL OF DATA STRUCTURE AND ALGORITHM FOR TECHNICIANS

MODULE CODE: COT 05213

MODULE COORDINATOR: DANIEL MADUHU

TASK: INDIVIDUAL ASSAIGNMENT 01.

01. To rank the following complexities in term of their rates of growth starting with slowest to fastest:
Given; O(n), O(log n), O(n^5), O(n log n), O(1)
In order to rank from slowest to fastest, we look at how the number of operations increases as the input size(n) grows.
Order from slowest to fastest is:

- O(1)
- O(log n)
- O(n)
- O(n log n)
- O(n^5)

02. To find the Big-O complexity

(a) $4n^2 + 20n + 9$
$=O(n^2)$
∴ The Big-O is n^2 because is the highest power

(b) $25n^3 + 15$
$=O(n^3)$
∴ The Big-O is n^3 because is the highest power

(c) $n^3 + 5\log n + 6$
$=O(n^3)$
∴The Big-O is n^3 since cubic is the highest power

(d) $10^5 + 10^5 n + 10^5 n^2$
$=O(n^2)$
∴The Big-O is n^2 because is the highest power

(e) $72n\ (n + 3n^2)$
$=72n^2 + 216n^3$
$=O(n^3)$
∴The Big-O is n^3

(f) $2^8 + 32^4 n - 5$
$=256 + 1048576n - 5$
$=O(n)$
∴The Big-O is n

03. To design an algorithm that run in a worst-case O(n) time to add all the numbers from 1 to a given number n (including n)

Solution

```
Algorithm sum (n)
Begin
  Sum=0
  For i = 1 to n do
    Sum = sum+1
End for
Print sum
End
```

Explanation; the algorithm starts by assigning the value 0 to variable sum. A loop is the used to iterate from one up to given number n. during each iteration, the current value of I is added to sum.

After the loop finishes, the final value of sum represents the addition of all numbers from 1 to n.

04. For each of the following code fragments, give the worst-case running time complexity in Big-O notation

(a) For this case give the complexity in term of the variable number.

```
X = 4
 For i: = 0 to number do:
  X: = x + 1
 End for
```

Explanation; the loops runs from 0 to number, so it executes approximately number + 1 times. Each operation inside the loop (x : = x + 1), takes constants time which is O(1). Therefore, the total running time grows linearly with number.

O(number)

T(n) = n + 1

= O(n)

(b) Complexity in term of cycles variable

Given;

```
Choice: = get _ choice ()
if choice = 1 then:
   printf "ready"
elseif choice = 2 then:
  for i: = 0 to cycle do:
         Print i
      end for
      endif
endif
```

solution

Explanation

Get _ choice () runs in O(1).

- If choice = 1, only one print statement executes O(1)
- If choice = 2, the loop runs cycles +1 times

In the worst case, the second condition occurs, so the loop dominates the running time. Hence the worst complexity is **O(cycles)**

(c) To find the complexity of the calculate function

Given, the two function below.

```
Increase (val: integer, iter:integer): integer
begin:
      for I: = 1 to iter do:
               val: = val + 5
     end for
    return val
    end
calculate (val:integer, inter:integer) : integer
begin:
     for i = 1 to iter do:
   val: = val +increase(val,iter)
   endfor
   return val
```

solution

Explanation;

Step 01: complexity of increase () function

The function increase () contains one loop, (for i: = 1 to iter do) this loop runs iter times. Each operation inside the loop takes constants time O (1), T(increase) = O(iter)

Therefore, the complexity of increase () is **O(iter)**

Step 02: complexity of calculate () function

The function calculate () also contains a loop, (for i: = 1 to iter do) also this loop executes iter times. Inside this loop, the function calls (increase (val, iter)).

Then from step one increase () = O(iter)

Therefore, for every iteration of calculate (), the program perform O(iter) work. Since the outer loop also runs iter times. So, the worst-case time complexity of the calculate function is **$O(iter^2)$**

(d) Find the complexity of the calculate method

Given;

```
increase (val: integer, iter: integer): integer
begin:
       for i: = 1 to iter do:
              val: val + 5
     endfor
     return val
end
calculate (val: integer, iter: integer) : integer
begin:
     for i: 1 to iter do:
          Val: = val + increase (val, i)
    endfor
    return val
end
```

Solution

Step 01: to analyze the increase () method

The method increase (val, iter) contains one loop (for i: = 1 to iter do), the loop executes iter times. Hence the running time of increase () is **O(iter)**

Step 02: to analyze the calculate method

The calculate () method contains: (for i: = 1 to iter do), this outer loop runs iter times. Inside the loop, the method calls (increase (val, i)) the second parameter is I, not iter

so, during each iteration;

- first iteration – increase (val,1) – O(n)
- second iteration – increase (val,2) – O(2)
- third iteration – increase (val,3) – O(3)
- last iteration – increase (val, iter) – O(iter)
- hence the total running time becomes 1+2+3+…….+iter
- by using summation formular = $\frac{iter\ (iter+1)}{2}$

Therefore, the worst-case time complexity of the calculate () method is O(iter2), because the outer loop runs iter times and inner method increase () executes an increasing number of operations from 1 up to iter.

05. To fill the blanks with either (≤, ≥, or =)

(i) For some functions f(n) and g(n), if f(n) is O(g(n)), then for some constants c and k, for all n ≥ k, **f(n) ≤ cg(n)**

(ii) For some function f(n) and g(n), if f(n) is Ω(g(n)), then for some constants c and k for all n ≥ k, **f(n) ≥ cg(n)**

06. Show that 4n + 12 is O(n). hint to find two suitable constants for c and k

Solution

First, we need to find the two possible constants, c and k, that satisfy the definition of Big O notation.

By definition, f(n) is O(g(n)) if there exist positive constants c and k such that: f(n) ≤ c. g(n) for all n ≥ k.

For this problem, f(n) = 4n + 12 and g(n) = n since we are dealing with growth rates, we assume n > 0, so we can drop the absolute bars: 4n + 12 ≤ c. n for all n ≥ k.

Step 01. We select a constants c that is strictly greater than the coefficient of our dominant term (4n). let's assume c = 5

Step 02. Substitute c = 5 into our definition inequality

4n + 12 ≤ 5n

Step 03. We solve for n to find k, subtract 4n from both sides of inequalities

12 ≤ 5n – 4n

12 ≤ n

This means the inequalities holds true for any value of n that is greater than or equal to 12

Therefore, our threshold constants k = 12

We found two suitable constants that are c = 5 and k = 12, because 4n + 12 ≤ 5n holds true for all n ≥ 12, we have proven that 4n + 12 is O(n)

07. Show that 5n2 – 16 is O(n2), hint find two suitable constants c and k, by letting c = 1 and solve inequality for that.

Solution

To show that $5n^2 – 16$ is $O(n^2)$, we need to find positive constants c and k such that the definition of Big-O holds true: $5n^2 – 16 \leq c.n2$ for all $n \geq k$

Step 01. We set c = 1, then the inequalities becomes, $5n^2 – 16 \leq 1.\ n2$

- $4n^2 \leq 16$
- $n^2 \leq 4$
- $n \leq 2$

note; the Big O definition requires the inequalities to hold for all n greater than or equal to k. here the inequality only holds when n is less than or equal to 2, therefore, c = 1 does not work because $5n^2$ grows much faster than $1n^2$. To make the inequality work for a large value of n, and c must be greater than or equal to 5

then, $5n^2 – 16 \leq 5n^2$

$= -16 \leq 0$

Since we found positive (such as c = 5, k = 1) that satisfy the inequality $5n^2 - 16 \leq c.\ n^2$ for all $n \geq k$, **we have rigorously shown that $5n^2 – 16$ is O(n)**